

```
`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 21.02.2026 11:47:08
// Design Name:
// Module Name: battery_monitor
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////

module battery_monitor(
    input clk,
    input rst,
    input [7:0] battery_value,
    output reg green_led,
    output reg yellow_led,
    output reg red_led,
    output reg buzzer
);

parameter SAFE_LEVEL = 8'd150;
parameter LOW_LEVEL = 8'd120;
parameter SAFE = 2'b00;
```

```

parameter LOW          = 2'b01;
parameter CRITICAL    = 2'b10;

reg [1:0] state, next_state;
always @(posedge clk or posedge rst)
begin
    if (rst)
        state <= SAFE;
    else
        state <= next_state;
end

always @(*)
begin
    next_state = SAFE;

    if (battery_value > SAFE_LEVEL)
        next_state = SAFE;

    else if (battery_value > LOW_LEVEL)
        next_state = LOW;

    else
        next_state = CRITICAL;
end

always @(*)
begin
    // Default outputs
    green_led  = 0;
    yellow_led = 0;
    red_led    = 0;
    buzzer     = 0;

    case(state)

        SAFE:
        begin
            green_led = 1;

```

```
end
```

```
LOW:
```

```
begin
```

```
    yellow_led = 1;
```

```
    buzzer      = 1;
```

```
end
```

```
CRITICAL:
```

```
begin
```

```
    red_led = 1;
```

```
    buzzer  = 1;
```

```
end
```

```
endcase
```

```
end
```

```
endmodule
```